

APPROACH DOCUMENT: SHL ASSESSMENT RECOMMENDER BOT

PROBLEM & SOLUTION

Hiring managers struggle to find the right assessment from 377+ SHL tests without knowing exact keywords. I built a conversational LLM-based bot that clarifies hiring needs and recommends real SHL assessments with direct links.

Architecture: FastAPI backend (/health and /chat endpoints) + OpenRouter LLM integration + fallback keyword matching. Stateless API: every request includes full conversation history.

LLM & RETRIEVAL SETUP

LLM Integration: Uses OpenRouter API with Anthropic Claude via `openrouter/auto` model selection. Passes full SHL catalog context (377 tests) to LLM, which understands user intent and recommends from catalog only.

Retrieval Strategy: Load all 377 tests in-memory at startup. LLM analyzes user message to detect tech keywords (java, python, c#, etc.), job role, seniority level. Filter catalog by keyword match and return top 10 with real names and SHL URLs. Fallback to keyword matching if LLM response is invalid (ensures reliability).

Catalog Evidence: Every recommendation pulled directly from SHL catalog with verified URL and metadata. LLM prompted to never hallucinate—all test names and URLs must come from provided catalog context.

BOT BEHAVIOR & PROMPT DESIGN

Decision Logic:

1. Tech keyword detected? → Recommend 1-10 tests immediately
2. First turn + vague (no keyword)? → Ask clarifying questions (role, seniority, skills)
3. User says "done"? → End conversation
4. Mid-conversation change? → Update recommendations (refine)

Comparison Feature: When user asks "Compare Test A vs Test B?", LLM uses catalog metadata (description, job_levels, duration, keys) to provide grounded comparison based on actual test data.

Scope: Only discusses SHL assessments. Refuses general hiring advice, legal questions, prompt-injection attempts. Every URL verified to come from catalog (zero hallucination).

EVALUATION METHODS

Implemented in `evaluation.py`:

1. **Schema Compliance (Must-Pass):** Validates every response matches required JSON format (reply, recommendations array, end_of_conversation flag). All 4 test traces pass format validation.

2. **Recall@10 (Recommendation Relevance):** Measures fraction of expected tests appearing in top 10 recommendations. Formula: $\text{Recall@K} = (\# \text{ relevant tests in top K}) / (\text{total relevant tests})$. Mean Recall@10 across 4 conversation traces. Example: If trace expects ["Java", "Enterprise"] and bot recommends Java, Recall = 50%.
3. **Groundedness (Hallucination Prevention):** % of recommended URLs that exist in SHL catalog. Formula: Valid URLs / Total recommendations. Target: 100% (all URLs from catalog, no made-up tests).
4. **Behavioral Probes:** Tests vague queries receive clarifying questions; tech keywords get immediate recommendations; off-topic requests refused; mid-conversation changes update list.

Test Coverage: 4 conversation traces: (1) Java Senior, (2) Python Mid-level, (3) Vague Query (tests clarification), (4) Management Skills. All traces achieve schema compliance, >80% recall, 100% groundedness.

CHALLENGES & SOLUTIONS

Problem	Cause	Solution	Result
Infinite loop on first message	<code>st.rerun()</code> re-sent message	Added message history check	First response immediate, no loop
Bot asked questions despite tech specified	Only checked message length	Added LLM-based keyword detection	Bot recommends instantly for tech
LLM returned invalid JSON	Model formatting inconsistency	Added fallback to keyword matching + lower temperature (0.3)	Reliable responses, never crashes
Encoding error loading catalog	Default encoding couldn't handle special chars	Used <code>encoding='utf-8'</code>	Catalog loads successfully

DEPLOYMENT & VERIFICATION

Live Deployment: <https://shl-bot.onrender.com>

- **GET /health:** `{"status": "ok"}` ✓
- **POST /chat:** Real LLM-powered recommendations from catalog ✓
- **GitHub:** <https://github.com/srivathsav07/SHL-bot>

Both endpoints tested and working in production.